

Tool Calling in LangGraph

Giving your chatbot the ability to act: custom and pre-built tools, ToolNode, tools_condition, and the feedback loop that turns raw tool output into a polished reply.

WHAT THIS CHAPTER COVERS

- ▶ Why a pure LLM chatbot cannot take real-world actions
- ▶ ToolNode and tools_condition — the two new building blocks
- ▶ Pre-built vs. custom tools (@tool decorator)
- ▶ Building a calculator, web search, and stock-price tool
- ▶ Binding tools to an LLM with bind_tools()
- ▶ The naive graph and why its output is unusable
- ▶ Fixing it with a tool → LLM feedback loop
- ▶ Integrating tools into the persistent chatbot project

Tool Calling in LangGraph

RECAP — CHAPTER 15

Chapter 15 added **observability** to the chatbot using LangSmith: every run was automatically traced once the four LangSmith environment variables were set, and adding `metadata.thread_id` to the invocation config grouped a user's turns into a single Thread. The chatbot itself — a single `chat_node` talking to an LLM, persisted with a SQLite checkpointer — was otherwise unchanged and still could not take any real-world action beyond generating text.

1. Why the Chatbot Needs Tools

Problem It Solves

Up to this point the chatbot has a GUI, short-term memory, database persistence, and streaming — but it is backed by a single LLM call, so it can only *talk*. If a user asks a question the model already knows the answer to (e.g. "What is the capital of India?"), a plain LLM call works fine. But if the user asks something that requires a real-world *action* — searching the live internet, performing an exact numeric calculation, or fetching a real-time stock price — the LLM has no way to do this on its own. It can only generate text based on its training data.

What This Chapter Builds

Three tools are added to the existing chatbot project:

1. **Calculator** — performs exact arithmetic (add / sub / mul / div) instead of relying on the LLM's unreliable mental math.
2. **Web Search** — uses DuckDuckGo to answer questions about current events the model was never trained on.
3. **Stock Price Lookup** — calls the Alpha Vantage API to fetch a live stock quote for a given ticker symbol.

🔄 Quick Revision — Why Tools

- A plain LLM node can only generate text; it cannot perform real actions.
- Tools extend an agent's capability beyond conversation into actions such as search, computation, and API calls.
- This chapter adds exactly three tools: a calculator, a web-search tool, and a stock-price tool.
- There is no limit to what a tool can wrap — any Python function or API call can become a tool.

2. The Two New Building Blocks: ToolNode and tools_condition

Definition — ToolNode

A *ToolNode* is a **prebuilt node** supplied by LangGraph (imported from `langgraph.prebuilt`) that acts as a bridge between the graph and a collection of external tools. Normally a LangGraph node is a function you write yourself — it takes in a state and returns a state. A ToolNode is a ready-made node that already knows how to handle a list of LangChain tools: its job is to listen for tool calls emitted by the LLM, automatically route each call to the correct tool (using the tool name and the input arguments the LLM produced), execute it, and place the tool's response back into the graph state.

Definition — tools_condition

tools_condition is a **prebuilt conditional-edge function** supplied by LangGraph. It is attached to the chat node's outgoing edge and inspects the LLM's response to decide where execution should go next: if the LLM's response contains a tool call, *tools_condition* routes the flow to the ToolNode; if the response is a normal answer with no tool call, it routes the flow straight to END.

IMPORTANT

ToolNode and *tools_condition* are two separate, complementary pieces. ToolNode **executes** whichever tool the LLM asked for. *tools_condition* **decides** whether a tool needs to be executed at all. Neither one does the other's job.

How It Works — Modifying the Basic Workflow

The simplest possible LangGraph workflow is `START → chat_node → END`, where `chat_node` holds a single LLM call. To support tools, this has to change in two ways:

1. The chat node must gain a decision-making ability: after reading the incoming messages, it must decide whether the user wants normal conversation or wants an action performed. This decision-making happens automatically once the LLM is *tool-aware* (see Section 3) — the LLM itself decides whether to reply normally or emit a tool call.
2. A new node — the ToolNode — must be added, holding the full collection of available tools (calculator, search, stock price). The chat node's outgoing edge becomes a *conditional edge* driven by *tools_condition*, so the graph can route either to the ToolNode or to END.

🔄 Quick Revision — ToolNode and tools_condition

- ToolNode is a prebuilt node that executes whichever tool the LLM's response requested.
- *tools_condition* is a prebuilt conditional-edge function that decides: go to the ToolNode, or go to END.
- ToolNode reads the tool name and input arguments directly from the LLM's tool-call output.
- Both are imported from `langgraph.prebuilt`.

3. Building the Tools

Definition — Pre-built vs. Custom Tools

LangChain tools come in two categories:

Type	What It Means	Example In This Chapter
Pre-built	Shipped by LangChain for common use cases — no need to write it yourself.	DuckDuckGoSearchRun
Custom	Written by the engineer for a project-specific need, using the <code>@tool</code> decorator.	<code>calculator</code> , <code>get_stock_price</code>

Implementation

All three tools are implemented in `backend.py`. Two rules apply to every custom tool built in this project, before looking at the code itself:

COMMON MISTAKE

Forgetting the docstring on a custom tool. The LLM reads the docstring to decide *which* tool fits a given task — a vague or missing docstring makes the LLM guess wrong. Every custom tool must clearly describe its exact purpose and expected inputs.

IMPORTANT

Never hard-code or reproduce a real API key. In `get_stock_price` below, the Alpha Vantage key is read from an environment variable (`os.environ['STOCK_API_KEY']`) — it is never written into the source code.

The pre-built search tool and the custom calculator tool:

```

from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool

# 1. Pre-built tool -- internet search
search_tool = DuckDuckGoSearchRun(region='us-en')

# 2. Custom tool -- exact arithmetic
@tool
def calculator(first_num: float, second_num: float, operation: str) -> dict:
    """
    Perform a basic arithmetic operation on two numbers,
    supported operations : add, sub, mul, div
    """
    try:
        if operation == 'add':
            result = first_num + second_num
        elif operation == 'sub':
            result = first_num - second_num
        elif operation == 'mul':
            result = first_num * second_num
        elif operation == 'div':
            if second_num == 0:
                return {'error': 'Division by zero is not allowed!'}
            else:
                result = first_num / second_num
        else:
            return {'error': f'Unsupported operation: {operation}'}

        return {'first_num': first_num, 'second_num': second_num,
                'operation': operation, 'result': result}
    except Exception as e:
        return {'error': str(e)}

```

The custom stock-price tool:

```

# 3. Custom tool -- live stock price via Alpha Vantage
@tool
def get_stock_price(symbol: str) -> dict:
    """
    Fetch latest stock price for a given symbol (e.g, 'AAPL','TSLA')
    Using Alpha Vantage with api key in the url.
    """
    api_key = os.environ['STOCK_API_KEY']
    url = (f"https://www.alphavantage.co/query?"
           f"function=GLOBAL_QUOTE&symbol={symbol}"
           f"&apikey=<YOUR_API_KEY>")
    r = requests.get(url)
    return r.json()

```

🔄 Quick Revision — Tools

- Pre-built tools (e.g. DuckDuckGoSearchRun) ship with LangChain for common needs.
- Custom tools are plain Python functions decorated with `@tool`.
- A tool's docstring is read by the LLM to decide when to use that tool -- always write a precise one.
- This project's three tools: `search_tool`, `calculator`, `get_stock_price`.

4. Making the LLM Tool-Aware

How It Works

Creating tools is not enough — the LLM itself must be told which tools exist and how to call them. This is done with `bind_tools()`, which returns a new, tool-aware version of the LLM:

```
# Collect all tools into a single list
tools = [search_tool, calculator, get_stock_price]

# Make the LLM tool-aware
llm_with_tools = llm.bind_tools(tools)
```

From this point on, the chat node must call `llm_with_tools`, never the plain `llm` — only the tool-aware version can decide to emit a tool call.

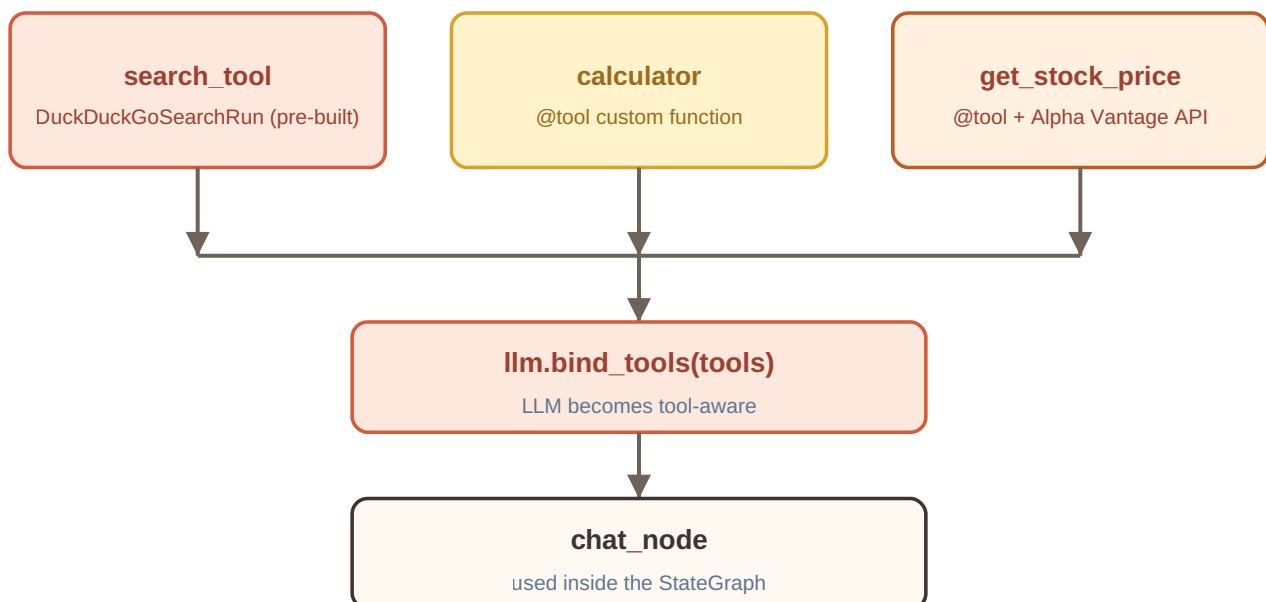


Figure 16.1 — Three tools are collected into a list and bound to the LLM with `bind_tools()`, producing a tool-aware LLM used inside `chat_node`.

🔄 Quick Revision — `bind_tools()`

- `bind_tools(tools)` returns a new, tool-aware LLM -- it does not modify the original `llm` object.
- `chat_node` must invoke `llm_with_tools`, not the plain `llm`, or tool calls will never be produced.
- The tools list passed to `bind_tools()` must be the same list wrapped inside the `ToolNode`.

5. The Naive Graph -- and Why It Is Broken

How It Works

With `ToolNode` and `tools_condition` available, the graph can be assembled: `chat_node` connects to the `ToolNode` through a conditional edge driven by `tools_condition`, and the `ToolNode` connects onward to `END`.

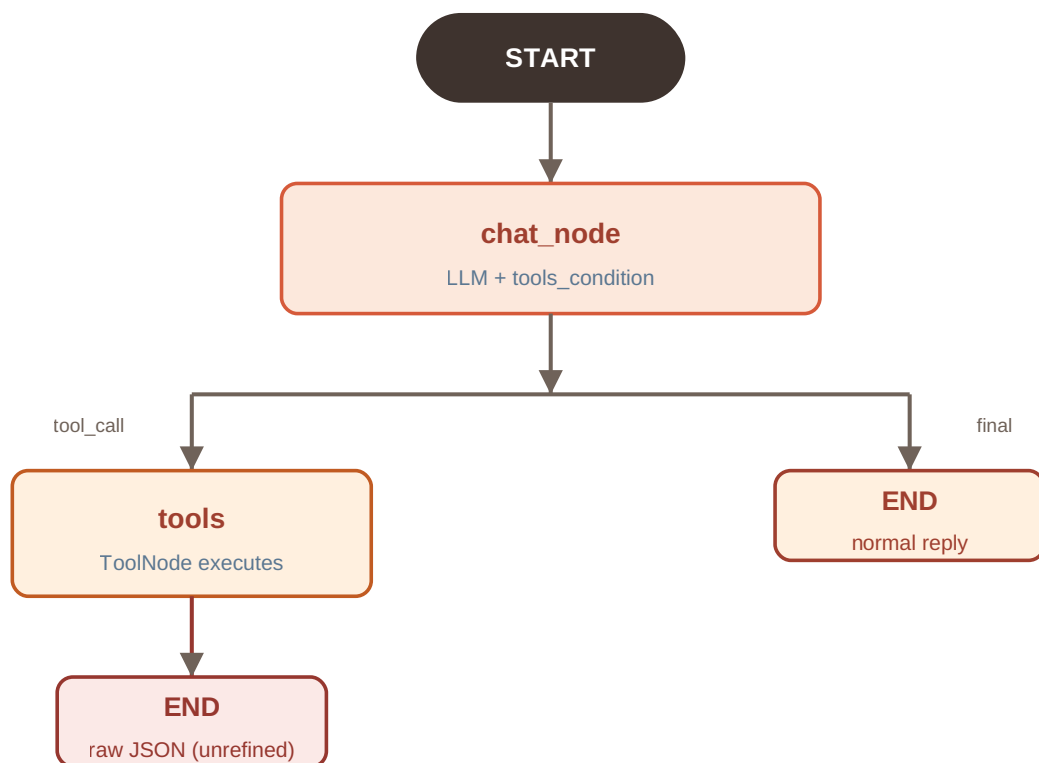


Figure 16.2 — A naive tool graph: `tools_condition` sends the flow to either `END` directly, or to the `tools` node -- which then dead-ends straight at its own `END`.

Problem It Solves — and the Problem It Creates

Running this graph confirms the basic mechanism works: a normal message like "Hello" flows straight to `END` with a normal reply, while a message like "What is 2 * 3?" is routed to the `tools` node and correctly triggers the calculator. But two serious problems appear as soon as real queries are tried:

1. UNREFINED, UNUSABLE OUTPUT

When asked "What is the stock price of Apple?", the graph's final output is the **raw JSON** returned directly by the Alpha Vantage API — a block of technical key-value data, not a sentence a normal user could read. Because the tool's output flows straight to END, there is no step left where the LLM can turn that raw data into a polished, human-readable sentence.

2. MULTI-STEP REASONING IS IMPOSSIBLE

A query such as "What is the stock price of Apple, how much will it cost me to buy 50 shares?" genuinely requires **two** tool calls in sequence: first look up Apple's price, then multiply it by 50. But because the ToolNode's output goes directly to END, the graph has no way to go back to the LLM after the first tool call to plan the second one. Running this query does not fail cleanly — it produces a nonsensical result (the calculator is invoked with meaningless inputs such as "50" and "0"), because the LLM never gets a chance to see the stock-lookup result before the graph terminates.

COMMON MISTAKE

Assuming that once a tool is added the graph is "done." Wiring ToolNode → END directly is a natural first attempt, but it silently breaks both output quality and any query that needs more than one tool call.

🔄 Quick Revision — The Naive Graph's Flaws

- chat_node → tools → END (direct) technically runs, but has two flaws.
- Flaw 1: the tool's raw output (e.g. JSON) is shown to the user unmodified and unpolished.
- Flaw 2: multi-step queries requiring two tools in sequence cannot be completed correctly.
- The root cause in both cases: the LLM never gets to see the tool's result before the graph ends.

6. The Fix: Looping the Tool Output Back to the LLM

How It Works

The fix is a single structural change: instead of connecting the ToolNode to END, connect it **back to chat_node**. This turns the tool node and the chat node into a small loop, letting the LLM see every tool result before deciding what happens next.

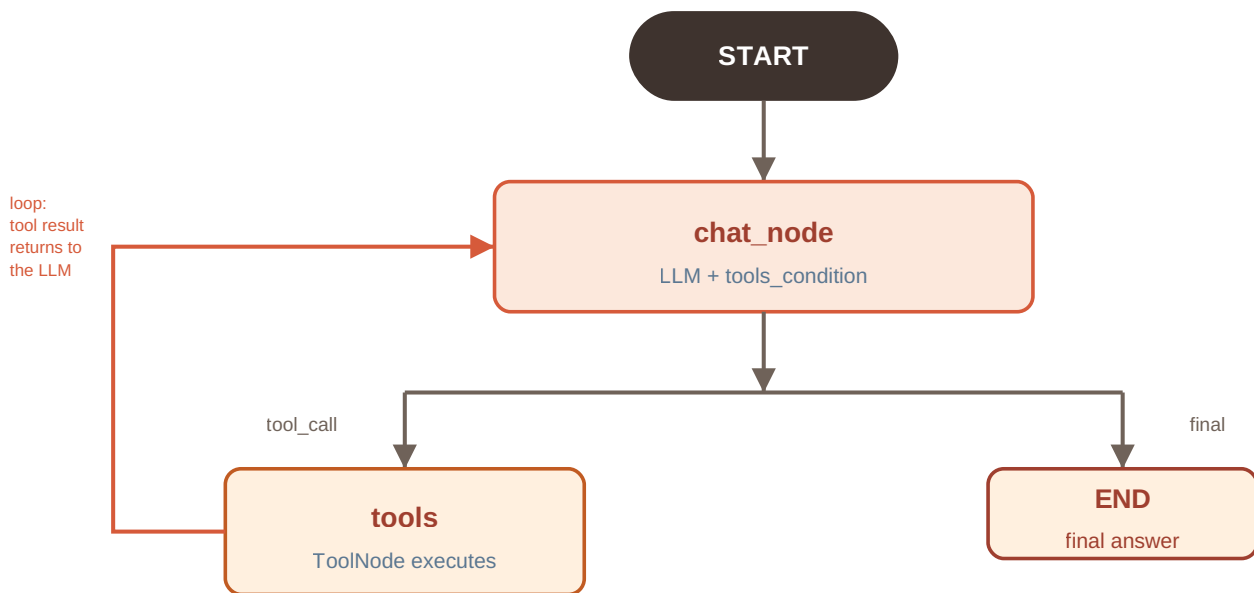


Figure 16.3 — The corrected graph: tool output always routes back to `chat_node`, which re-evaluates the full history before answering or calling another tool.

```
graph = StateGraph(MessageState)

graph.add_node('chat_node', chat_node)
graph.add_node('tools', tool_node)

graph.add_edge(START, 'chat_node')

# If the LLM asks for a tool, go to tool_node; otherwise finish
graph.add_conditional_edges('chat_node', tools_condition)

# Send the tool's response back to the LLM so it can decide
# whether another tool call is needed, or the final answer is ready
graph.add_edge('tools', 'chat_node')
```

Example — How the Loop Resolves Both Problems

With the loop in place:

- **"What is 2 * 3?"** — the calculator returns 6, `chat_node` receives that result and replies in plain language: "The result of 2 * 3 is 6."
- **"What is the stock price of Apple?"** — the raw JSON returns to `chat_node`, which reads it and produces a clean sentence stating the price, instead of dumping the JSON.
- **"Stock price of Apple, then cost of 50 shares"** — `chat_node` first calls the stock tool, receives the price, reasons over the full conversation history (which now includes that result), and issues a *second* tool call to the calculator with the correct numbers before producing the final answer.

IMPORTANT

`tools_condition` still governs the *outgoing* edge from `chat_node` (tool vs. END) exactly as before. The only structural change is the *return* edge from tools back to `chat_node` — this is what turns a one-shot tool call into an arbitrarily deep reasoning loop.

INTERVIEW TIP

If asked to explain agentic tool use in an interview, describe it as a loop, not a pipeline: LLM → (optional) tool → LLM → (optional) tool → ... → END. The LLM re-evaluates the entire message history after every tool call, which is what allows multi-step plans to emerge without being hard-coded.

🔄 Quick Revision — The Fixed Graph

- The only structural change: `add_edge('tools', 'chat_node')` replaces `tools -> END`.
- `tools_condition` is unchanged -- it still decides `chat_node`'s outgoing route.
- The loop lets the LLM see and reason over every tool result before answering or calling another tool.
- This single change fixes both the raw-output problem and the multi-step reasoning problem.

7. Tracing the Loop in LangSmith

Real World Insight

Because the chatbot project already has LangSmith tracing enabled (Chapter 15), the tool loop can be inspected directly. For the query "What is the stock price of Apple?", the trace shows three distinct steps: first, control reaches `chat_node` with the three tools available, and the LLM decides to call `get_stock_price` with the input "Apple"; `tools_condition`'s output for this step is `tools`, so control moves to the `ToolNode`. Second, the `ToolNode` calls `get_stock_price` with the resolved ticker and receives the raw API response. Third, control returns to `chat_node`, which now has the full history — the original question, its own tool-call decision, and the tool's result — and produces the final answer; at this point `tools_condition`'s output is `END`.

REAL WORLD INSIGHT

Visualizing a trace like this in LangSmith is one of the fastest ways to build an accurate mental model of when the LLM is being called versus when a tool is being called, especially while debugging a multi-step tool chain that isn't behaving as expected.

🔄 Quick Revision — Tracing Tool Loops

- A tool-using turn appears in LangSmith as multiple runs: `chat_node -> tools -> chat_node`.
- Each `chat_node` run shows `tools_condition`'s decision for that step (tools or END).
- Tracing makes it easy to confirm exactly when the LLM decided to call a tool versus produce a final answer.

8. Integrating Tools Into the Chatbot Project

Implementation — backend.py

The existing chatbot backend is rewritten to include the tool graph, while keeping every previously built feature: the message state, the SQLite checkpointer, and the `retrieve_all_threads()` helper. The full graph-assembly section, exactly as implemented:

```
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool

# ... tool definitions and llm_with_tools as shown in Section 3 & 4 ...

class MessageState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

def chat_node(state: MessageState):
    """LLM node that may answer a question or request a tool call."""
    messages = state['messages']
    response = llm_with_tools.invoke(messages)
    return {'messages': [response]}

tool_node = ToolNode(tools)

graph = StateGraph(MessageState)
graph.add_node('chat_node', chat_node)
graph.add_node('tools', tool_node)
graph.add_edge(START, 'chat_node')
graph.add_conditional_edges('chat_node', tools_condition)
graph.add_edge('tools', 'chat_node')

# Persistence layer -- unchanged from Chapter 14
conn = sqlite3.connect(database='chatbot.db', check_same_thread=False)
checkpointer = SqliteSaver(conn=conn)
chatbot = graph.compile(checkpointer=checkpointer)
```

COMMON MISTAKE

The lecture demonstrates `ChatOpenAI` during the live walkthrough, but the project's actual attached `backend.py` uses `ChatNVIDIA(model="nvidia/nemotron-3.5-lightning-30b-a3b")` as the underlying LLM. Per this project's source-precedence rule, the attached implementation file is authoritative — the notes above reflect the actual code, not the spoken walkthrough.

Implementation — frontend.py Wiring Change

The Streamlit frontend requires only one change: the import line is updated to point at the new backend module containing the tool-enabled graph. No other frontend logic needs to change for the tool loop itself to work end-to-end.

```
from backend import chatbot, retrieve_all_threads
```

🔄 Quick Revision — Backend Integration

- The tool graph replaces the old single-node graph inside `backend.py`; state, checkpointer, and helpers are unchanged.
- `chat_node` now invokes `llm_with_tools` instead of a plain `llm`.
- `frontend.py` needs only its import statement updated to use the new backend.
- Attached source code (ChatNVIDIA) takes precedence over the lecture's spoken example (ChatOpenAI).

9. Streaming Only the Final Answer

Problem It Solves

Once tools are wired into the streaming chatbot, a new UX issue appears: `chatbot.stream()` emits **every** message chunk produced inside the graph, including the `ToolMessage` carrying the tool's raw result. If the frontend streams every chunk it receives, the tool's raw output momentarily flashes on screen before the LLM's polished reply appears — exactly the unpolished-output problem the loop in Section 6 was meant to solve, just re-introduced at the streaming layer.

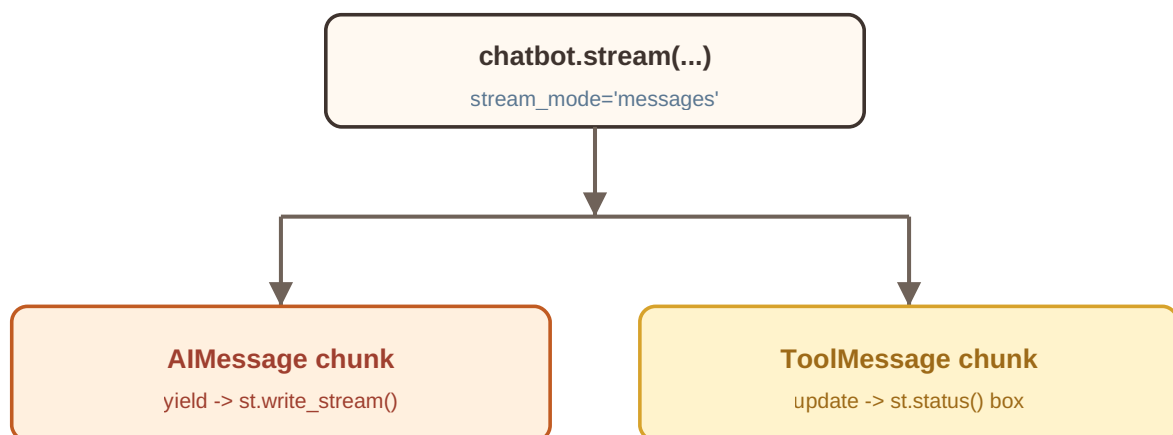


Figure 16.4 — `chatbot.stream()` emits both `AIMessage` and `ToolMessage` chunks; the frontend must stream only `AIMessage` chunks to the user.

Implementation

The fix is a type check on each streamed chunk: only chunks that are an `AIMessage` are yielded to `st.write_stream()`; a `ToolMessage` chunk is instead used only to update a status indicator, exactly as implemented in `frontend.py`:

```

from langchain_core.messages import HumanMessage, AIMessage, ToolMessage

def ai_only_stream():
    for message_chunk, metadata in chatbot.stream(
        {'messages': [HumanMessage(content=user_input)]},
        config=CONFIG,
        stream_mode='messages',
    ):
        # Lazily create / update a status box whenever a tool runs
        if isinstance(message_chunk, ToolMessage):
            tool_name = getattr(message_chunk, 'name', 'tool')
            if status_holder['box'] is None:
                status_holder['box'] = st.status(f"Using `{tool_name}` ...", expanded=True)
            else:
                status_holder['box'].update(
                    label=f"Using `{tool_name}` ...",
                    state='running', expanded=True)

        # Stream ONLY assistant tokens
        if isinstance(message_chunk, AIMessage):
            yield message_chunk.content

```

Best Practice — the Status Container

Streamlit's chat interface has four chat-element types; this project uses a **status container** (`st.status()`) as the fourth. Whenever a `ToolMessage` chunk is seen, the status box is created or updated to show which tool is currently running (e.g. "Using `duckduckgo_search` ..."), then marked complete once the run finishes. This keeps the user informed about whether the chatbot is chatting normally or performing an action, without ever streaming the tool's raw output as if it were the final reply.

BEST PRACTICE

Always filter streamed chunks by message type in a tool-using agent. Treat `AIMessage` chunks as user-facing text and treat `ToolMessage` (and any other non-AI) chunks as internal signals for UI state, never as text to render directly.

🔄 Quick Revision — Streaming Filter

- `chatbot.stream(..., stream_mode='messages')` yields both `AIMessage` and `ToolMessage` chunks.
- Only `AIMessage` chunks should be yielded to `st.write_stream()`; `ToolMessage` chunks are UI signals, not user-facing text.
- A status container (`st.status()`) is used to show which tool is currently running.
- This is the fourth Streamlit chat element used in this project, alongside chat messages, `chat_input`, and `write_stream`.

K. Key Takeaways

- **Tools** extend an LLM-only chatbot into an agent capable of real-world actions: calculation, web search, and API calls.
- **ToolNode** is a prebuilt node that executes whichever tool the LLM's response requested, using the tool name and arguments the LLM produced.
- **tools_condition** is a prebuilt conditional-edge function that routes chat_node's output to either the ToolNode or END.
- **Pre-built vs. custom tools** — DuckDuckGoSearchRun is pre-built; calculator and get_stock_price are custom tools built with the @tool decorator, each requiring a precise docstring.
- **bind_tools()** produces a tool-aware LLM; chat_node must call this version, not the plain LLM.
- **The naive graph** (tools → END) technically runs but produces unrefined raw output and cannot support multi-step tool use.
- **The fix** is a single feedback edge, tools → chat_node, turning the graph into a loop where the LLM sees every tool result before deciding what happens next.
- **Streaming** must filter by message type: only AIMessage chunks go to the user; ToolMessage chunks drive a status container instead.

R. Revision Sheet

ToolNode

Prebuilt LangGraph node that executes tool calls requested by the LLM and returns their results into the graph state.

bind_tools()

Returns a new, tool-aware LLM object; the original LLM is left unmodified.

tools_condition

Prebuilt conditional-edge function that routes chat_node's output to the ToolNode (if a tool call exists) or to END.

The Tool → LLM Loop

add_edge('tools', 'chat_node') sends every tool result back to the LLM, enabling polished answers and multi-step tool chains.

@tool decorator

Turns a plain Python function into a custom LangChain tool; the function's docstring tells the LLM when to use it.

Stream Filtering

Only AIMessage chunks are streamed as visible text; ToolMessage chunks update a status indicator instead.

I. Interview Questions

- Q1.** What is the difference between a ToolNode and a regular LangGraph node that you write yourself?
- Q2.** What decision does tools_condition make, and what are its two possible outputs?
- Q3.** What is the difference between a pre-built tool and a custom tool? Give one example of each from this chapter.
- Q4.** Why is the docstring on a custom @tool function important, beyond documentation purposes?
- Q5.** What does bind_tools() actually return, and why must chat_node call that object instead of the plain LLM?
- Q6.** Describe the two concrete problems with a graph that routes tools directly to END.
- Q7.** What single change fixes both problems in the naive tool graph, and why does it work?
- Q8.** Walk through what happens, step by step, when a chatbot with this tool loop is asked a question requiring two sequential tool calls.
- Q9.** Why can streaming every message chunk from chatbot.stream() produce a poor user experience in a tool-using agent, and how is this fixed?
- Q10.** In a LangSmith trace of a tool-using turn, what would you expect to see, and in what order?

F. Further Reading

- LangGraph documentation — Prebuilt ToolNode and tools_condition reference
- LangChain documentation — the @tool decorator and custom tool creation
- LangChain documentation — DuckDuckGoSearchRun and other community search tools
- Alpha Vantage API documentation — GLOBAL_QUOTE endpoint reference
- Streamlit documentation — st.status() and chat elements